Day 36 Practice Problems: Deep Learning Foundations & Perceptrons

This practice suite contains three production-grade engineering challenges designed to solidify your mastery of artificial neuron mathematics, non-linear decision boundaries, PyTorch tensor forward passes, and deep learning framework architectures taught in Day 36.

Challenge 1: Manual Forward Pass & PyTorch Verification of a Diagnostic Neuron

Background & Context

You are developing a binary risk stratification model for cardiovascular triage. A single artificial neuron processes four normalized patient physiological signals:

1. Normalized Age ($x_1 = 0.60$)
2. Normalized Resting Blood Pressure ($x_2 = 0.75$)
3. Normalized Serum Cholesterol ($x_3 = 0.80$)
4. Normalized Maximum Heart Rate ($x_4 = 0.50$)

The pre-trained synaptic weight vector and bias assigned to this neuron are:

$$\mathbf{w} = \begin{bmatrix} 0.40 \\ 0.70 \\ 0.50 \\ -0.60 \end{bmatrix}, \quad b = -0.15$$

Requirements & Tasks

1. **Manual Linear Combination (z)**:
   - Write out the algebraic summation equation and calculate the exact scalar value of the weighted sum:

$$z = \sum_{i=1}^{4} w_i x_i + b$$

2. **Sigmoid Probability Activation ($\hat{y}$)**:
    - Pass the calculated linear sum z through the Sigmoid activation function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

    - Compute the final predicted probability $\hat{y}$ (rounded to 4 decimal places). State whether the patient is classified as high-risk if threshold $\tau = 0.50$.
3. **PyTorch Verification Implementation**:
    - Implement this exact neuron in PyTorch using `torch.nn.Linear(in_features=4, out_features=1)`.
    - Manually set the layer's `.weight` and `.bias` tensors to the exact values above.
    - Run the forward pass with input tensor `torch.tensor([0.60, 0.75, 0.80, 0.50])`, apply `torch.sigmoid()`, and verify that PyTorch's output matches your manual calculation to 4 decimal places.

Challenge 2: Non-Linearity & Overcoming the XOR Perceptron Limitation

Background & Context

In 1969, Minsky and Papert demonstrated that a single-layer perceptron is incapable of solving the **XOR (Exclusive OR)** function because the truth table cannot be separated by a single straight linear line.

XOR Truth Table:

Input x_1	Input x_2	Target Output y	Point (x_1, x_2)
0	0	0	$(0, 0)$
0	1	1	$(0, 1)$
1	0	1	$(1, 0)$
1	1	0	$(1, 1)$

Requirements & Tasks

1. **Mathematical Impossibility Proof**:

- Set up the four simultaneous linear inequalities:

$$w_1(0) + w_2(0) + b < 0 \implies b < 0$$

$$w_1(0) + w_2(1) + b \geq 0 \implies w_2 + b \geq 0$$

$$w_1(1) + w_2(0) + b \geq 0 \implies w_1 + b \geq 0$$

$$w_1(1) + w_2(1) + b < 0 \implies w_1 + w_2 + b < 0$$

- Show algebraically why this system of inequalities produces a mathematical contradiction, proving that no single perceptron can solve XOR.

2. **Multi-Layer Architecture Design (MLP)**:
   - Design a 2-layer Multi-Layer Perceptron with 2 input neurons (x_1, x_2), 2 hidden neurons (h_1, h_2) using ReLU or Step activation, and 1 output neuron (y) that decomposes XOR into fundamental logic gates (e.g., OR, NAND, AND).
   - Specify the exact weights and biases for all neurons in your network.

Challenge 3: Industrial Architecture Sizing & Framework Governance

Background & Context

A machine vision company is building an automated defect inspection pipeline across 10 million industrial component images. The system must process high-resolution $1024 \times 1024 \times 3$ RGB images in real time.

Requirements & Tasks

1. **Traditional ML vs Deep Learning Decision Framework**:
   - Provide three concrete engineering reasons why traditional computer vision feature extractors (e.g., SIFT, HOG, edge filters combined with an SVM classifier) fail on this task compared to a deep convolutional/vision transformer architecture.
2. **Framework Selection Strategy**:
   - Evaluate whether the engineering team should select **PyTorch**, **TensorFlow**, or **Keras** given that the R&D team requires rapid experimentation with custom

attention modules, while the deployment team requires low-latency deployment on edge NVIDIA Jetson hardware.

3. **Hardware & VRAM Memory Budgeting**:
    - Calculate the raw uncompressed memory footprint of a single mini-batch of 32 images of shape $(32, 3, 1024, 1024)$ using 32-bit floating point precision (`float32` = 4 bytes per float).
    - Explain why training requires 3x to 4x more VRAM than the raw batch data alone (hint: forward activations, backward gradients, optimizer states).

Solutions Reference

For complete step-by-step mathematical proofs, PyTorch verification code, and industrial sizing breakdowns, review Solutions.md.